

Yesterday 11:21 AM

VibeCodingGitRep...
SpreadsheetLab_4_VibeCoding...
PDF

my number is 35, so tell me everything i have to do from start to end
to finish this task

Worked for 18s

Analyzed ▾

</> Python

```
import openpyxl, os, json, textwrap, pandas as pd
p="/mnt/data/VibeCodingGitRepoLinks(1).xlsx"
wb=openpyxl.load_workbook(p,data_only=True)
wb.sheetnames
```

Result

```
['VibeCodingRepos']
```

Analyzed ▾

</> Python

```
ws=wb['VibeCodingRepos']
rows=list(ws.iter_rows(values_only=True))
rows[:8], ws.max_row, ws.max_column
```

Result

```
([('Sl. No', 'Git Repo Link'),
 (1, 'https://github.com/SETAPESU26/01_frogger'),
 (2, 'https://github.com/SETAPESU26/02_air_hockey'),
 (3, 'https://github.com/SETAPESU26/03_fishing'),
 (4, 'https://github.com/SETAPESU26/04_target_shooting'),
 (5, 'https://github.com/SETAPESU26/05_balloon_pop'),
 (6, 'https://github.com/SETAPESU26/06_coin_collector'),
 (7, 'https://github.com/SETAPESU26/07_catch_falling')],
 65,
 2)
```

Analyzed ▾

 Python

```
[r for r in rows if r[0]==35]
```

Result

```
[(35, 'https://github.com/SETAPESU26/35_qbert')]
```

Your assigned repo is **#35 — Q*bert Repair Lab**. The assignment sheet says you must work individually, run the original Python game, record a 10-second “before” video, use AI prompts to fix/add features, record a 10-second “after” video, push your updated code with **separate commits for each task**, and submit everything under your Lab-4 folder. Also, **do not raise a PR to SETAPESU26**.

 Lab_4_VibeCoding_Student_handout

Your repo is:

[SETAPESU26/35_qbert on GitHub](#) ↗

The repo README confirms that it is a Pygame Q*bert game with **1 deliberate bug + 3 functions/features to implement**.  GitHub

Do this from start → finish

1. Fork the repo FIRST

Open the repo above while logged into GitHub.

Click **Fork** → **Create fork**.

You should end up with something like:

https://github.com/YOUR_USERNAME/35_qbert

This is important because the lab specifically says to push the code to **your repo** and **not** raise a PR to the original SETAPESU26 repo.  Lab_4_VibeCoding_Student_handout

2. Clone YOUR fork

Open VS Code.

Open a terminal and run:

 Bash

```
git clone https://github.com/YOUR_USERNAME/35_qbert.git
cd 35_qbert
```

Replace `YOUR_USERNAME` with your GitHub username.

Then open it:

 Bash

```
code .
```

You should see:

 Plain text

```
35_qbert
├── game.py
├── README.md
└── .gitignore
```

The README confirms `game.py` is the actual game code.  [GitHub](#)

3. Install Pygame

Check Python:

 Bash

```
python --version
```

The repo asks for Python **3.10+**. Then install Pygame:

 Bash

```
pip install pygame
```

If that doesn't work:

 Bash

```
python -m pip install pygame
```

The repo specifically lists Pygame as the required dependency.  [GitHub](#)

4. Run the BROKEN game

Do not change anything yet.

Run:

 Bash

```
python game.py
```

The controls are:

 Plain text

← ↑ ↓ →	Move/hop
R	Reset
Space	Continue after clearing level

At this point, the pyramid should look **skewed instead of symmetrical**. That's intentional — Task 1 is to fix that projection bug.  [GitHub](#)

5. RECORD YOUR BEFORE VIDEO 🚨

This is easy to forget.

While the game is still untouched, record around **10 seconds** showing the broken/skewed pyramid and some gameplay.

Save it as something obvious like:

 Plain text

```
before.mp4
```

The lab sheet explicitly requires a 10-second video **before making changes**.  [Lab_4_VibeCoding_Student_handout](#)

Now start the actual coding

You have **4 tasks**.

Don't ask AI to do all four in one giant prompt. The assignment wants iterative prompting and separate commits.

6. TASK 1 — Fix the pyramid projection bug

The README basically gives you the debugging direction.

The problem is inside:

 Python

```
cube_center
```

The arithmetic converting (row, col) into the cube's screen position is wrong. Specifically, you need to investigate whether the row offset should use **integer division** or **true division**. The finished pyramid must be a symmetric triangle.  [GitHub](#)

Use a prompt like this in ChatGPT:

 Plain text

```
I have a Pygame Q*bert game for a lab assignment.
```

```
Task 1 says the pyramid is visibly skewed. Each cube's screen position is calculated from (row, col) coordinat
```

```
The README hints that the problem is related to whether the row offset uses integer division or true division.
```

```
Please inspect my game.py and identify the exact bug. Explain why it causes the pyramid to become asymmetric a
```

```
Do not change unrelated functionality.
```

Then upload/paste `game.py` .

After making the fix:

 Bash

```
python game.py
```

Verify the pyramid is now symmetrical.

Then commit **only Task 1**:

`</>` Bash

```
git add game.py
git commit -m "Task 1: Fix pyramid projection bug"
```

7. TASK 2 — Implement `cube_palette(level)`

Find:

`</>` Python

```
cube_palette(level)
```

Right now it's an empty/incomplete feature.

The README says it should return **3 RGB colors** because `TARGET + 1` is currently 3.

Basically:

`</>` Plain text

```
Stage 0 → unpainted
Stage 1 → partially painted
Stage 2 → fully painted
```

The README suggests using a different 3-color palette for each level.  [GitHub](#)

Use this as your second AI prompt:

`</>` Plain text

```
Task 1 is complete.
```

```
Now implement Task 2 only.
```

```
Implement cube_palette(level) in my Q*bert Pygame game.
```

```
The function should return TARGET + 1 colors, currently 3 RGB tuples, representing:
```

1. unpainted cube
2. partially painted cube
3. fully painted cube

Use different visually distinct palettes depending on the level.

Keep the existing game behavior unchanged and give me the complete updated game.py.

Run:

`</>` Bash

```
python game.py
```

Make sure cube colors change correctly as you land on them.

Then:

`</>` Bash

```
git add game.py
git commit -m "Task 2: Implement level cube palettes"
```

8. TASK 3 — Implement `on_cube_completed(cell)`

This function gets called when a cube **first reaches its final painted stage**.

The README suggests either:

- briefly flash the completed cube, **or**
- award a small bonus.

Since the existing game already gives **25 points per paint step**, a small completion bonus is probably the simpler implementation.  [GitHub](#)

Your next prompt:

`</>` Plain text

Tasks 1 and 2 are complete.

Now implement Task 3 only: `on_cube_completed(cell)`.

This function is called only when a cube first reaches its final target color.

Implement a simple and clearly visible/useful completion feature, such as awarding a small bonus score when a

Do not interfere with the existing 25 points awarded for each paint step.

Keep all previous fixes and features working and give me the complete updated game.py.

Run:

 Bash

```
python game.py
```

Test that finishing a cube gives the bonus.

Then:

 Bash

```
git add game.py
git commit -m "Task 3: Add cube completion bonus"
```

9. TASK 4 — Bonus life

Find:

 Python

```
bonus_life_threshold()
```

The README suggests:

 Python

```
return 1000
```

That means the player gets a bonus life whenever their score crosses another multiple of **1000**.

The bookkeeping for preventing duplicate rewards is already implemented by the game, so you shouldn't rewrite that system.  [GitHub](#)

Prompt:

`</>` Plain text

Tasks 1, 2 and 3 are complete.

Now implement Task 4 only: `bonus_life_threshold()`.

The README suggests a bonus life threshold of 1000 points.

The game's existing `self.bonus_awarded` logic already handles awarding the life only once for each threshold cr

Implement this without modifying unrelated game logic and give me the complete updated `game.py`.

Test:

`</>` Bash

```
python game.py
```

Then:

`</>` Bash

```
git add game.py
git commit -m "Task 4: Add bonus life threshold"
```

10. Test EVERYTHING

Before recording the final video, check all expected behavior.

The repo says the final game should have:

- symmetrical pyramid
- cube advances exactly one color stage per landing
- falling off the pyramid costs a life
- Coily chases the player rather than moving away
- completing every cube wins the level
- running out of lives ends the game
- your new cube palettes work

- your cube-completion feature works
- bonus life system works  GitHub

Also test:

 Plain text

Arrow keys
R
Space

Don't record the final video until the game runs without errors.

11. Check your commits

Run:

 Bash

```
git log --oneline
```

You should see something roughly like:

 Plain text

```
xxxxxxx Task 4: Add bonus life threshold
xxxxxxx Task 3: Add cube completion bonus
xxxxxxx Task 2: Implement level cube palettes
xxxxxxx Task 1: Fix pyramid projection bug
xxxxxxx Initial repo commit
```

This matters because your handout explicitly requires **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

12. Record the AFTER video

Now record another ~10-second video.

Try to clearly show:

 Plain text

```
Symmetrical pyramid  
+  
player hopping  
+  
cubes changing colors  
+  
your added feature
```

Save it as:

 Plain text

```
after.mp4
```

The assignment requires both before and after recordings.



Lab_4_VibeCoding_Student_handout

13. Push everything to YOUR GitHub

Run:

 Bash

```
git status
```

Ideally:

 Plain text

```
nothing to commit, working tree clean
```

Then:

 Bash

```
git push origin main
```

Go to your GitHub fork and confirm that `game.py` has changed and all four task commits appear.

Do not create a Pull Request back to SETAPESU26. That's explicitly prohibited.

 Lab_4_VibeCoding_Student_handout

14. Save/export your AI chat

This is also part of the submission.

Your lab handout asks for:

 Plain text

Chat history exported as doc/pdf

 Lab_4_VibeCoding_Student_handout

So keep this conversation and all the prompts you use while fixing the code. Your repo README similarly asks for the complete LLM/chat history.  [GitHub](#)

15. Make your Lab-4 folder

Your final submission should basically look like:

 Plain text

```
Lab-4/
|
├─ before.mp4
├─ after.mp4
├─ 35_qbert/
|   └─ game.py
|   └─ README.md
|   └─ .gitignore
└─ chat_history.pdf
```

Your handout specifically says the Lab-4 folder must contain the **before/after videos, updated code, and exported chat history.**  Lab_4_VibeCoding_Student_handout

The order I recommend you follow right now

Fork → Clone → Install pygame → Run original → RECORD BEFORE → Task 1 → Commit → Task 2 → Commit → Task 3 → Commit → Task 4 → Commit → Full test → RECORD AFTER → Push → Export chat → Put everything in Lab-4 → Submit.

One important thing: **don't modify `game.py` before taking that first video.** Once you've recorded the broken version, send me your `game.py` (or tell me you've reached Task 1), and I can walk you through each task one at a time so your commits and AI chat history cleanly match the assignment. </>

my git hub username is **Adi911-hack**, give the codes wrt this

Yep. With your GitHub username, use these exact commands:

</> Bash

```
git clone https://github.com/Adi911-hack/35_qbert.git
cd 35_qbert
code .
```

After completing Task 1:

</> Bash

```
git add game.py
git commit -m "Task 1: Fix pyramid projection bug"
```

After Task 2:

</> Bash

```
git add game.py
git commit -m "Task 2: Implement level cube palettes"
```

After Task 3:

</> Bash

```
git add game.py
git commit -m "Task 3: Add cube completion bonus"
```

After Task 4:

 Bash

```
git add game.py
git commit -m "Task 4: Add bonus life threshold"
```

Then check everything:

 Bash

```
git status
git log --oneline
```

Finally push to your GitHub:

 Bash

```
git push origin main
```

So your repo should be:

 Plain text

```
https://github.com/Adi911-hack/35\_qbert
```

Important: that URL will only work **after you fork repo #35 into your `Adi911-hack` account**. The assignment requires you to clone/fork your assigned repo, push the modified code to your personal repo, and keep separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

If you haven't forked it yet, do that first. Then we can go **Task 1 → Task 4 together**, including the exact `game.py` code changes and exactly when you should record/commit/push.

this is the repo  https://github.com/Adi911-hack/lab_4_335_qbert,
give me codes wrt this

Yep — I checked your actual repo. It is `Adi911-hack/lab_4_335_qbert`, it's on the `main` branch, and it currently contains `game.py`, `README.md`, and `.gitignore`. It's correctly forked from `SETAPESU26/35_qbert`.  [GitHub](#)

Use these commands from now on.

1. Clone your actual repo

Open **VS Code** → **Terminal** and run:

 **Bash**

```
git clone https://github.com/Adi911-hack/lab_4_335_qbert.git
cd lab_4_335_qbert
code .
```

Your repo: [Adi911-hack/lab_4_335_qbert](https://github.com/Adi911-hack/lab_4_335_qbert) ↗

2. Install Pygame

 **Bash**

```
python -m pip install pygame
```

Then run:

 **Bash**

```
python game.py
```

Your README requires Python 3.10+ and Pygame.  [GitHub](#)

Do NOT edit `game.py` yet. First record the required **10-second BEFORE video** showing the skewed pyramid. Your README specifically requires this.  [GitHub](#)

3. After recording, do Task 1

The first task is fixing the skewed pyramid inside `cube_center` . The README specifically points to the calculation involving `row` , `col` , and **integer division vs true division**.  [GitHub](#)

After you make that change and verify it works:

 **Bash**

```
git add game.py
git commit -m "Task 1: Fix pyramid projection bug"
```

4. Task 2

Implement:

 Python

```
cube_palette(level)
```

It needs to return **3 RGB colors** representing the three painting stages.  [GitHub](#)

After testing:

 Bash

```
git add game.py
git commit -m "Task 2: Implement cube palette"
```

5. Task 3

Implement:

 Python

```
on_cube_completed(cell)
```

The README says this runs when a cube reaches its final stage for the **first time** and suggests something like a small bonus beyond the existing 25 points.  [GitHub](#)

After testing:

 Bash

```
git add game.py
git commit -m "Task 3: Add cube completion feature"
```

6. Task 4

Implement:

 Python

```
bonus_life_threshold()
```

The README explicitly suggests:

```
</> Python
```

```
return 1000
```

The existing `self.bonus_awarded` code handles the rest.  [GitHub](#)

Then:

```
</> Bash
```

```
git add game.py
git commit -m "Task 4: Add bonus life threshold"
```

7. Check your four commits

```
</> Bash
```

```
git log --oneline
```

You want to see your four tasks separately, because the lab handout requires **separate commits for each task**.  [Lab_4_VibeCoding_Student_handout](#)

Then check:

```
</> Bash
```

```
git status
```

8. Push to YOUR repo

Because your remote came from cloning `Adi911-hack/lab_4_335_qbert` , this is all you need:

```
</> Bash
```

```
git push origin main
```

You **do not** need:

```
</> Bash
```

```
git remote add origin ...
```

because `origin` is already set by `git clone`.

9. Final test + video

Run:

 Bash

```
python game.py
```

Check that the pyramid is symmetrical, cubes change stages correctly, falling costs a life, Coily chases you, completing all cubes wins the level, your palette works, cube completion feature works, and bonus lives work. These are the expected behaviours in your README.  [GitHub](#)

Then record your **10-second AFTER video**.

10. Final submission

Your lab handout requires your **before video**, **after video**, **updated code**, and **exported AI chat history**.



[Lab_4_VibeCoding_Student_handout](#) Your repo README also specifically lists the before video, after video, and complete Chat/LLM history link.  [GitHub](#)

So the workflow is:

Clone → **install** → **run** → **BEFORE VIDEO** → **Task 1** → **commit** → **Task 2** → **commit** → **Task 3** → **commit** → **Task 4** → **commit** → **test** → **AFTER VIDEO** → **push** → **submit chat history**.

And **don't create a Pull Request** to `SETAPESU26`; your lab handout explicitly says not to.



[Lab_4_VibeCoding_Student_handout](#)

If you want to start coding now, the next thing is **Task 1**. I can give you the exact change to make in your current `game.py`, then we commit it before touching Task 2.

Yesterday 10:25 PM

i want a more slower steo by step, take me through is slowly

Yep. We'll do it **one tiny step at a time**.

For now, do **only Step 1**.

Step 1 — Open VS Code

Open **VS Code**.

Then at the top, click:

Terminal → New Terminal

A terminal should open at the bottom.

Once you see the terminal, type this exact command:

```
</> Bash
```

```
git clone https://github.com/Adi911-hack/lab_4_335_qbert.git
```

Then press **Enter**.

That's it for now.

After it finishes, send me exactly what your terminal shows, and I'll give you **Step 2 only**.

no no, giveme every step rn

Worked for 47s

Got it. Here's the **full start-to-finish version**, but broken into tiny steps. Follow it in order and **do not jump ahead**, especially because you need separate Git commits for every task.

Your actual repo is `Adi911-hack/lab_4_335_qbert` , and I verified it is forked from the assigned `SETAPESU26/35_qbert` .  [GitHub](#) Your lab requires a before video, AI-assisted changes, separate commits, an after video, pushing to your repo, and your chat history.  [Lab_4_VibeCoding_Student_handout](#)

1. **Open VS Code.** Go to **Terminal → New Terminal**. You should now see a terminal at the bottom.
2. **Clone your repo.** Paste:

```
</> Bash
```

```
git clone https://github.com/Adi911-hack/lab_4_335_qbert.git
```

Press Enter. Then enter the folder:

```
</> Bash
```

```
cd lab_4_335_qbert
```

Then open that folder in VS Code:

```
</> Bash
```

```
code .
```

If VS Code asks whether you trust the authors, click **Yes, I trust the authors**.

3. Check that Git is connected correctly. In the VS Code terminal run:

```
</> Bash
```

```
git remote -v
```

You should see something containing:

```
</> Plain text
```

```
https://github.com/Adi911-hack/lab_4_335_qbert.git
```

Now check your branch:

```
</> Bash
```

```
git branch
```

You should see:

```
</> Plain text
```

```
* main
```

4. Check Python. Run:

```
</> Bash
```

```
python --version
```

Your repo asks for Python 3.10 or newer.  GitHub

If `python` doesn't work on Windows, try:

```
</> Bash
```

```
py --version
```

5. Install Pygame. Run:

```
</> Bash
```

```
python -m pip install pygame
```

If you're using `py`, use:

```
</> Bash
```

```
py -m pip install pygame
```

Your README specifically requires Pygame.  [GitHub](#)

6. Run the ORIGINAL game before touching the code. Run:

```
</> Bash
```

```
python game.py
```

or, if needed:

```
</> Bash
```

```
py game.py
```

A Q*bert window should open.

Use the arrow keys to move.

At this point the pyramid should look weird/skewed. That's the intentional bug. The README says the pyramid should be symmetrical but currently drifts sideways.  [GitHub](#)

Do not change any code yet.

7. **Record your BEFORE video now.** 📹 Record around **10 seconds** of the game while the broken pyramid is visible. Move Q*bert around a little so it clearly shows gameplay.

Save the file somewhere safe as:

📄 Plain text

before.mp4

This is compulsory. Your handout specifically says to record 10 seconds **before making any changes**.

📄 Lab_4_VibeCoding_Student_handout

Once that video is saved, close the game.

TASK 1 — Fix the pyramid

8. **Open game.py** . In VS Code, click:

📄 Plain text

game.py

Near the top you'll find this exact function:

📄 Python

```
def cube_center(row, col):  
    return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
```

I verified that's what your current repo contains.  GitHub

The problem is:

📄 Python

```
row // 2
```

// performs integer division. On odd numbered rows that destroys the .5 offset needed to center the pyramid.

9. **Change Task 1 only.** Replace:

`</>` Python

```
def cube_center(row, col):  
    return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
```

with:

`</>` Python

```
def cube_center(row, col):  
    return pygame.Vector2(WIDTH / 2 + (col - row / 2) * CUBE_W, 90 + row * CUBE_H)
```

The only real change is:

`</>` Plain text

```
//
```

became:

`</>` Plain text

```
/
```

Save with **Ctrl + S**.

10. Test Task 1. Run:

`</>` Bash

```
python game.py
```

The pyramid should now look like a properly centered symmetrical triangle.

Close the game.

11. Commit Task 1 separately. Run:

`</>` Bash

```
git status
```

You should see `game.py` modified.

Then:

 Bash

```
git add game.py
```

Then:

 Bash

```
git commit -m "Task 1: Fix pyramid projection bug"
```

Do not do Task 2 before making this commit. Your handout requires separate commits for each task.

 [Lab_4_VibeCoding_Student_handout](#)

TASK 2 — Add cube colours

12. Go back to the top of `game.py` .

You currently have:

 Python

```
def cube_palette(level):  
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""  
    pass
```

Your README says this function must return `TARGET + 1` colors. `TARGET` is currently `2` , meaning we need **3 colors**: unpainted, partially painted, and fully painted.  [GitHub](#)

Replace that whole function with:

 Python

```
def cube_palette(level):  
    palettes = [  
        [(90, 160, 220), (190, 120, 70), (100, 210, 140)],  
        [(120, 90, 200), (230, 140, 70), (250, 220, 90)],  
        [(70, 160, 150), (220, 90, 130), (120, 220, 100)]
```

```
]
return palettes[(level - 1) % len(palettes)]
```

So that part of the file should now look like:

Python

```
DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}

def cube_palette(level):
    palettes = [
        [(90, 160, 220), (190, 120, 70), (100, 210, 140)],
        [(120, 90, 200), (230, 140, 70), (250, 220, 90)],
        [(70, 160, 150), (220, 90, 130), (120, 220, 100)]
    ]
    return palettes[(level - 1) % len(palettes)]
```

Save with **Ctrl + S**.

13. Test Task 2.

Bash

```
python game.py
```

Move Q*bert onto cubes.

Every time you correctly land on a cube it should move through its colour stages.

Your existing game already handles painting and awards 25 points per stage; we're only supplying the palettes here.  [GitHub](#)

Close the game.

14. Commit Task 2.

Bash

```
git add game.py
git commit -m "Task 2: Implement cube palette"
```

Now you have two separate commits.

TASK 3 — Flash a completed cube

15. Your current file contains:

 Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pass
```

The README says this function is called **only when a cube reaches its final stage for the first time**. It suggests things such as a brief flash or a bonus.  [GitHub](#)

We're going to make the cube briefly **flash white**.

First, go to the section near:

 Python

```
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}
```

Immediately underneath it, add:

 Python

```
FLASH_DURATION = 350  
CUBE_FLASHES = {}
```

So you now have:

 Python

```
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}  
  
FLASH_DURATION = 350  
CUBE_FLASHES = {}
```

Now find:

 Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pass
```

Replace it with:

 Python

```
def on_cube_completed(cell):  
    CUBE_FLASHES[cell] = pygame.time.get_ticks() + FLASH_DURATION
```

But we still need to actually **draw** the flash.

Scroll down and find the function:

 Python

```
def draw_cube(self, screen, cell, colors):
```

Inside it you'll see this near the end:

 Python

```
pygame.draw.polygon(screen, color, top)  
pygame.draw.polygon(screen, (240, 240, 240), top, 1)
```

Immediately **after those two lines**, add:

 Python

```
flash_until = CUBE_FLASHES.get(cell)  
  
if flash_until:  
    if pygame.time.get_ticks() < flash_until:  
        pygame.draw.polygon(screen, (255, 255, 255), top, 4)  
    else:  
        CUBE_FLASHES.pop(cell, None)
```

Therefore, that ending section should look like:

 Python

```

pygame.draw.polygon(screen, shade(DEFAULT_PALETTE[0], 0.45), left)
pygame.draw.polygon(screen, shade(DEFAULT_PALETTE[0], 0.3), right)
pygame.draw.polygon(screen, color, top)
pygame.draw.polygon(screen, (240, 240, 240), top, 1)

flash_until = CUBE_FLASHES.get(cell)

if flash_until:
    if pygame.time.get_ticks() < flash_until:
        pygame.draw.polygon(screen, (255, 255, 255), top, 4)
    else:
        CUBE_FLASHES.pop(cell, None)

```

Save.

16. Test Task 3.

 Bash

```
python game.py
```

A cube needs **two paint stages** to reach the target because your current file has:

 Python

```
TARGET = 2
```

So when you finally complete a cube, you should see a brief white outline flash.

Your existing `paint()` function already calls `on_cube_completed(cell)` when `self.stages[cell] == TARGET`, so we don't need to modify `paint()`.  [GitHub](#)

Close the game.

17. Commit Task 3.

 Bash

```
git add game.py
git commit -m "Task 3: Add cube completion flash"
```

TASK 4 — Bonus life

18. Near the top you currently have:

 Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    pass
```

Replace it with:

 Python

```
def bonus_life_threshold():  
    return 1000
```

That's literally all Task 4 needs.

Your README specifically suggests a threshold of `1000`, and the existing game already contains the bookkeeping that checks whether the score has crossed a threshold and adds a life.  [GitHub](#) +1

Save.

19. Test Task 4.

 Bash

```
python game.py
```

The system will give an extra life when the score reaches another multiple of 1000.

You do **not** need to rewrite the life-awarding code.

Close the game.

20. Commit Task 4.

 Bash

```
git add game.py  
git commit -m "Task 4: Add bonus life threshold"
```

CHECK THAT YOUR GIT HISTORY IS CORRECT

21. Run:

`</> Bash`

```
git log --oneline
```

Near the top you should see something similar to:

`</> Plain text`

```
xxxxxxx Task 4: Add bonus life threshold
xxxxxxx Task 3: Add cube completion flash
xxxxxxx Task 2: Implement cube palette
xxxxxxx Task 1: Fix pyramid projection bug
```

The random-looking xxxxxxxx values are normal Git commit IDs.

Then run:

`</> Bash`

```
git status
```

Ideally it says:

`</> Plain text`

```
nothing to commit, working tree clean
```

FINAL GAME TEST

22. Run the final game again:

`</> Bash`

```
python game.py
```

Check everything carefully.

The pyramid should be symmetrical.

The arrow keys should move Q*bert diagonally.

Landing on a cube should advance its color one stage.

Completing a cube should produce your brief flash.

Going off the pyramid should cost a life.

Coily should chase the player.

Completing the entire pyramid should clear the level.

The next level should use another palette.

At 1000-point score thresholds, the bonus-life system should work.

These match the expected behavior described in your repo.  GitHub

RECORD THE AFTER VIDEO

23. Start screen recording.

Run:

 Bash

```
python game.py
```

Record around **10 seconds** showing the corrected symmetrical pyramid and gameplay.

Try to show Q*bert changing a cube's colors.

Save it as:

 Plain text

```
after.mp4
```

Your repo checklist explicitly requests a before video and an after video showing the corrected/new behaviour.  GitHub

PUSH EVERYTHING TO YOUR GITHUB

24. First check:

 Bash

```
git status
```

Then push:

 Bash

```
git push origin main
```

Because you cloned:

 Plain text

```
Adi911-hack/lab_4_335_qbert
```

this pushes to **your repo**, not SETAPESU26.

Open your repo in your browser afterward and refresh it:

Your [lab_4_335_qbert repository](#) ↗

Click **Commits** and make sure your Task 1, Task 2, Task 3, and Task 4 commits are visible.

Do not create a Pull Request to SETAPESU26. Your lab handout explicitly tells you not to.

 [Lab_4_VibeCoding_Student_handout](#)

Your final `game.py` changes

After all four tasks, the relevant top section should essentially contain:

Python

```

DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}

FLASH_DURATION = 350
CUBE_FLASHES = {}

def cube_palette(level):
    palettes = [
        [(90, 160, 220), (190, 120, 70), (100, 210, 140)],
        [(120, 90, 200), (230, 140, 70), (250, 220, 90)],
        [(70, 160, 150), (220, 90, 130), (120, 220, 100)]
    ]
    return palettes[(level - 1) % len(palettes)]

def on_cube_completed(cell):
    CUBE_FLASHES[cell] = pygame.time.get_ticks() + FLASH_DURATION

def bonus_life_threshold():
    return 1000

def cube_center(row, col):

```

And inside `draw_cube()` you will have:

Python

```

pygame.draw.polygon(screen, color, top)
pygame.draw.polygon(screen, (240, 240, 240), top, 1)

flash_until = CUBE_FLASHES.get(cell)

if flash_until:
    if pygame.time.get_ticks() < flash_until:
        pygame.draw.polygon(screen, (255, 255, 255), top, 4)
    else:
        CUBE_FLASHES.pop(cell, None)

```

Don't paste all of this at once before committing. Make Task 1 → commit, Task 2 → commit, Task 3 → commit, Task 4 → commit. Otherwise you won't satisfy the separate-commit requirement.

Submission at the very end

Your handout says the Lab-4 submission must contain the **before video**, **after video**, **updated code**, and **exported chat history**.  Lab_4_VibeCoding_Student_handout Your specific repo README also says to submit the two videos and your complete Chat/LLM history link.  GitHub

So before you're completely done, make sure you have:

</> Plain text

before.mp4
after.mp4
updated game.py / pushed GitHub repo
this complete ChatGPT chat history

And again: no PR to the original SETAPESU26 repo.



this part has error, give full pastable code for this

Yep — the error is from **indentation**. Those `pygame.draw...` lines and the flash code need to stay inside `draw_cube()`.

Replace your **entire** `draw_cube()` function with this exact code:

</> Python

```
def draw_cube(self, screen, cell, colors):
    cx, cy = cube_center(*cell)

    top = [
        (cx, cy - CUBE_H / 2),
        (cx + CUBE_W / 2, cy),
        (cx, cy + CUBE_H / 2),
        (cx - CUBE_W / 2, cy)
    ]

    color = colors[self.stages[cell]]

    left = [
        top[3],
        top[2],
        (cx, cy + CUBE_H / 2 + SIDE),
        (cx - CUBE_W / 2, cy + SIDE)
    ]

    right = [
        top[1],
```

Also make sure you already have this near the top of `game.py` :

`</>` Python

```
FLASH_DURATION = 350  
CUBE_FLASHES = {}
```

And:

`</>` Python

```
def on_cube_completed(cell):  
    CUBE_FLASHES[cell] = pygame.time.get_ticks() + FLASH_DURATION
```

Then save with:

`</>` Plain text

Ctrl + S

and run:

`</>` Bash

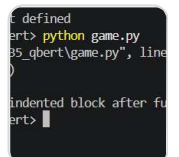
```
python game.py
```

The issue in your screenshot is basically that these lines:

`</>` Python

```
pygame.draw.polygon(...)  
flash_until = ...
```

were pushed all the way to the left, so Python thought they were **outside the function**. The code above fixes that.



Yep, this is 100% an **indentation problem** now.

Python is telling you:

</> Plain text

```
IndentationError: expected an indented block after function definition
```

So your file probably looks like this:

</> Python

```
def draw_cube(self, screen, cell, colors):
    cx, cy = cube_center(*cell)
```

But it must look like this:

</> Python

```
def draw_cube(self, screen, cell, colors):
    cx, cy = cube_center(*cell)
```

Everything inside the function needs to be indented.

Replace your **entire draw_cube function** with this exact code:

</> Python

```
def draw_cube(self, screen, cell, colors):
    cx, cy = cube_center(*cell)

    top = [
        (cx, cy - CUBE_H / 2),
        (cx + CUBE_W / 2, cy),
        (cx, cy + CUBE_H / 2),
        (cx - CUBE_W / 2, cy)
    ]

    color = colors[self.stages[cell]]

    left = [
        top[3],
        top[2],
        (cx, cy + CUBE_H / 2 + SIDE),
        (cx - CUBE_W / 2, cy + SIDE)
```

]

right = [

Important: make sure there are **4 spaces** before every line inside the function.

Then save:

 Plain text

Ctrl + S

and run:

 Bash

python game.py

If you still get an error, send me a screenshot showing **around lines 165–205** in `game.py`, because then the issue will likely be the function being placed at the wrong indentation level inside the class.



Yep — now the problem is clearer.

`draw_cube()` is a method **inside** **class Game:**, so:

- `def draw_cube(...)` needs **4 spaces**
- everything inside it needs **8 spaces**

Right now, parts like `(cx, cy - CUBE_H / 2)` are sitting at the class level, so Python tries to evaluate `cx` before the function even runs.

Replace your entire `draw_cube` section with this exact version:

 Python

```
def draw_cube(self, screen, cell, colors):
    cx, cy = cube_center(*cell)

    top = [
        (cx, cy - CUBE_H / 2),
```

```

        (cx + CUBE_W / 2, cy),
        (cx, cy + CUBE_H / 2),
        (cx - CUBE_W / 2, cy)
    ]

    color = colors[self.stages[cell]]

    left = [
        top[3],
        top[2],
        (cx, cy + CUBE_H / 2 + SIDE),
        (cx - CUBE_W / 2, cy + SIDE)
    ]

    right = [
        top[1],

```

Very important: the first line starts with **4 spaces**:

 Python

```
def draw_cube(...)
```

And this line has **8 spaces**:

 Python

```
cx, cy = cube_center(*cell)
```

Then save:

 Plain text

```
Ctrl + S
```

and run:

 Bash

```
python game.py
```

Your current `NameError: cx is not defined` should disappear once the whole method is indented like this.



i committed task 3 2 times by mistake

Yep — this is actually easy to fix, and since your screenshot says your branch is **ahead of origin/main by 4 commits**, you haven't pushed them yet. So we can clean it up safely.

From your screenshot, the likely situation is:

 Plain text

Task 1

Task 2

Task 3 ← correct

Task 3 ← this second one probably contains your Task 4 change

That's why when you tried to commit Task 4, Git said:

 Plain text

```
nothing to commit, working tree clean
```

So don't delete anything yet.

First run:

Bash

```
git show --stat --oneline HEAD
```

If that latest commit is the one where you changed:

Python

```
def bonus_life_threshold():
    return 1000
```

then just rename that latest commit from Task 3 to Task 4.

Run:

 Bash

```
git commit --amend -m "Task 4: Add bonus life threshold"
```

Then check:

 Bash

```
git log --oneline -5
```

You should now see something like:

 Plain text

```
xxxxxxx Task 4: Add bonus life threshold  
cbe8203 Task 3: Add cube completion flash  
xxxxxxx Task 2: Implement cube palette  
xxxxxxx Task 1: Fix pyramid projection bug
```

That is exactly what you want.

Then run:

 Bash

```
git status
```

It should still say:

 Plain text

```
nothing to commit, working tree clean
```

After that, you're good to continue to the final testing/video/push step.

Do not run `git reset` right now. Based on your screenshot, the second Task 3 commit most likely just needs its message renamed to Task 4.



Yep — this confirms it.

Your latest commit:

`</>` Plain text

```
8330609 Task 3: Add cube completion flash
1 file changed, 1 insertion(+), 2 deletions(-)
```

is almost certainly actually **Task 4**, because changing:

`</>` Python

```
def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
    pass
```

to:

`</>` Python

```
def bonus_life_threshold():
    return 1000
```

is exactly **1 insertion and 2 deletions**.

So just rename the latest commit. Run:

`</>` Bash

```
git commit --amend -m "Task 4: Add bonus life threshold"
```

Then check:

`</>` Bash

```
git log --oneline -5
```

You should now see something like:

</> Plain text

```
8330609 Task 4: Add bonus life threshold
cbe8203 Task 3: Add cube completion flash
xxxxxxx Task 2: Implement cube palette
xxxxxxx Task 1: Fix pyramid projection bug
```

Then run:

</> Bash

```
git status
```

If it says:

</> Plain text

```
nothing to commit, working tree clean
```

you're good.

After that, the next step is:

</> Bash

```
git push origin main
```

But **don't push until you check the `git log --oneline -5` output once**, just to make sure all 4 task names look correct.

Today 10:44 AM

so one final deliverable is this chat history, how do i upload a pdf of this chat history

Yep. Your handout specifically requires the **chat history exported as a DOC/PDF**.

 Lab_4_VibeCoding_Student_handout

The easiest way to submit **this exact conversation** is:

1. Open this chat on your laptop/browser.
2. Press **Ctrl + P**.
3. In the print window, choose **Save as PDF** as the printer/destination.
4. Save it as something clear, for example:
`Lab4_Chat_History.pdf`
5. Open the saved PDF once and scroll through it to make sure the prompts, code, and responses are actually visible.
6. Put that PDF in your **Lab-4 folder** along with `before.mp4` , `after.mp4` , and your updated code/repo materials.
7. Upload that Lab-4 folder/files wherever your instructor has told you to submit them.

One thing: the print-to-PDF result can sometimes cut off or format long chats badly. Check the PDF before submitting it.

If your instructor specifically wants the chat history **inside your GitHub repo**, then copy `Lab4_Chat_History.pdf` into your local `lab_4_335_qbert` folder and run:

`</>` Bash

```
git add Lab4_Chat_History.pdf
git commit -m "Add Lab 4 chat history"
git push origin main
```

That submission commit is fine—it doesn't interfere with your four separate task commits.